COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	September 1, 2026	**DATE SUBMITTED:**	September 9, 2026
NAME:	Uchi, Mao Sophie S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

Table 6-1. Data Generated and Observations.

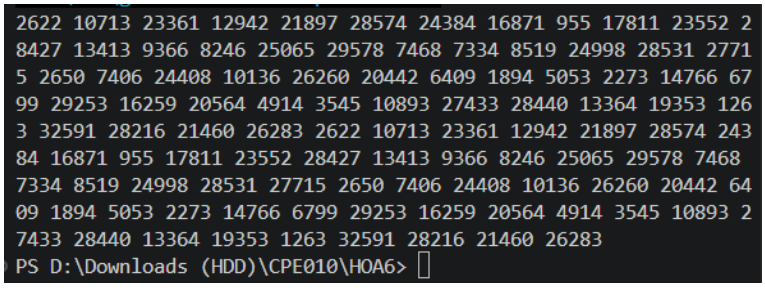
Screenshot	
Observations	With cstdlib & time.h library, we can generate random integers using the function rand(), yet they may seem random but that depends on the given seed. Instead of rand(), we use srand() for a different seed, and to avoid repetition outputting the same random set of values.

Table 6-2a. Linear Search for Arrays

Code

```
#include <iostream>
#include <cstdlib> //For generating randon int
#include <time.h>  //Will be used for our seeding funciton
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    //Generate random values
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i]=rand();
        std::cout << dataset[i] << " ";
    }

    int choice;
    std::cout << "Indicate the number to search: ";
    std::cin >> choice;

    linearSearchArr(max_size, choice, dataset);
}
```

```cpp
template <typename T>
void linearSearchArr(int maxSize, T dataFind, T data[]){
    int i = 0;
    while(i <= maxSize){
        if(dataFind == data[i]){
            std::cout << "Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout << "the search is unsuccessful\n";

}
```

Output

```
1175 529 6621 2494 7293 25737 21824 2934 26353 21661 23538 8914
3523 19376 7518 12003 11223 4326 15525 7315 14201 18987 28051 81
43 26353 7800 7633 2028 17239 19793 15964 13123 23977 24467 1669
3 26143 23067 21868 8027 2266 17838 30123 31478 4418 27140 8762
12757 6977 4979 21992
Indicate the number to search: 4979
Searching is successful
```

Observations

With Linear Search Array, we are able to search for a target element from the random numerical list, but how? Each element in the array is compared to each other, until the element is found, if not, it may continue to loop until the search for the element is unsuccessful. The Best Case for this algorithm is O(1), as searching for the target element might be found at the first index while the Worst Case for this algorithm is O(n) as searching for the target element might be found last or it might not exist from the array.

Table 6-2b. Linear Search for Linked List

Code

```
50  template <typename T>
51  void linearSearchLL(Node<T> *node, T dataFind) {
52      while(node != nullptr){
53
54          if(node->data == dataFind){
55              std::cout << "the search was successful.\n";
56              return;
57          }
58          node = node->next;
59      }
60      std::cout << "the search was unsuccessful\n";
61  }
```

```
 8   int main() {
 9
10       //Create linked list for linear search
11       Node<char>* name1 = new_node('U');
12       Node<char>* name2 = new_node('C');
13       Node<char>* name3 = new_node('H');
14       Node<char>* name4 = new_node('I');
15
16       //Linked List
17       name1->next = name2;
18       name2->next = name3;
19       name3->next = name4;
20       name4->next = nullptr;
21
22       //Linear Search
23       linearSearchLL(name1, 'M');
24       linearSearchLL(name2, 'C');
25       linearSearchLL(name3, 'U');
26       linearSearchLL(name4, 'I');
27   }
```

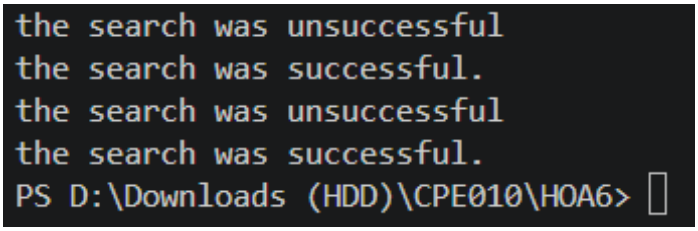
Output	
Observations	With Linear Search Linked List, we are able to search for a target element from the random numerical list, but how? Similar to Linear Search Array, both sequentially compare the elements from one another, hence this algorithm Best Case is still O(1) and Worst Case is still O(n).

Table 6-3a. Binary Search for Arrays

Code

```
 8  int main() {
 9
10      int dataset2[max_size];
11      for(int i = 0; i < max_size; i++){
12          dataset2[i] = i;
13          std::cout << dataset2[i] << " ";
14      }
15
16      binarySearchArr(11, dataset2, max_size);
17      binarySearchArr(11111, dataset2, max_size);
18
19  }
```

```
50  template <typename T>
51  void binarySearchArr(T dataFind, T data[], int maxSize){
52      int low = 0, up = maxSize-1, middle;
53      while(low <= up){
54          middle = (int)((low+up)/2);
55          if(dataFind == data[middle]){
56              std::cout << "\nSearching is successful\n";
57              return;
58          }
59          else if(dataFind > data[middle]){
60              low = middle + 1;
61          }else {
62              up = middle - 1;
63          }
64      }
65      std::cout << "Search element was not found.\n";
66  }
```

```cpp
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Output

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45
46 47 48 49
Searching is successful
Search element was not found.
PS D:\Downloads (HDD)\CPE010\HOA6>
```

Observations

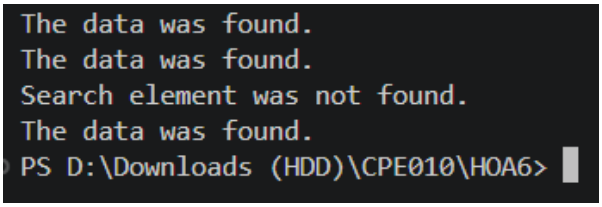
With Binary Search Array, we are able to search for a target element from the random numerical list, but how? Before progressing into the code, we must remember for our binary search to work, we must rearrange the array element in numerical order. We start at the middle of the array to check whether the target element is greater or lesser, with lesser continuing to the left side of the half array and with greater continuing to the right side of the array. This loop will continue until the target element is found. The Best Case for this algorithm is O(1), as searching for the target element might be found at the middle index while the Worst Case for this algorithm is O(log n) as searching for the target element might be found last or it might not exist from the array.

Table 6-3b. Binary Search for Linked List

Code

```
8   int main() {
9
10      Node<char>* name6 = new_node('A');
11      Node<char>* name7 = new_node('B');
12      Node<char>* name8 = new_node('C');
13      Node<char>* name9 = new_node('D');
14
15      name6->next = name7;
16      name7->next = name8;
17      name8->next = name9;
18      name9->next = nullptr;
19
20      binarySearchLL(name6, 'C');
21      binarySearchLL(name7, 'B');
22      binarySearchLL(name8, 'A');
23      binarySearchLL(name9, 'D');
24
25  }
```

```
87  template <typename T>
88  void binarySearchLL(Node<T>* start, T dataFind){
89
90      Node<T>* last = nullptr;
91
92      while (start != last) {
93          Node<T>* middle = getMiddle(start, last);
94
95          if(middle->data == dataFind) {
96              std::cout << "The data was found.\n";
97              return;
98          }
99          else if(middle->data > dataFind) {
100             last = middle;
101         } else {
102             start = middle->next;
103         }
104     }
105     std::cout << "Search element was not found.\n";
106 }
```

Output	
Observations	With Binary Search Linked List, we are able to search for a target element from the random numerical list, but how? Similar to Binary Search Array, both logarithmically compare the elements from one another, hence this algorithm Best Case is still O(1) and Worst Case is still O(log n).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1: Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Code

ilo_b.cpp > ...

```cpp
const int max_size = 10;

int main() {

    int dataset[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    Node<int>* node = new_node(dataset[0]);
    Node<int>* tail = node;
    std::cout << "ARRAY AND LINKED LIST LINEAR SEARCH\n";
    for(int i = 1; i < max_size; ++i){
        tail->next = new_node(dataset[i]);
        tail = tail->next;
    }

    linearSearchArr(max_size, 18, dataset);
    linearSearchLL(node, 18);
}
```

```cpp
template<typename T>
void linearSearchArr(int size, T dataFind, T data[]){
    int compare = 0;
    for (int i=0; i<size; i++) {
        std::cout << "Comparing: " << ++compare << "\n";
        if(data[i] == dataFind){
            std::cout << "Element Found at Index: " << i << "\n";
            return;
        }
    }
    std::cout << "Element not Found.\n";
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T> *node, T dataFinding) {
    int compare = 0;
    while(node != nullptr){
        std::cout << "Comparing: " << ++compare << "\n";
        if(node->data == dataFinding){
            std::cout << "Search was successful.\n";
            return;
        }
        node = node->next;
    }
    std::cout << "Search was unsuccessful\n";
}
```

Output

```
ARRAY AND LINKED LIST LINEAR SEARCH
Comparing: 1
Comparing: 2
Element Found at Index: 1
Comparing: 1
Comparing: 2
Search was successful.
PS D:\Downloads (HDD)\CPE010\HOA6>
```

"Identify how many comparisons would be necessary to find the key '18'?"

Using Linear Search, we will always start at the first element for comparison, but the element '15'(first index) is not a match to the target element '18'. We continue the search by comparing the second element '18'(second index) to the target element '18'; the comparison is a success as both elements are of equal value. From the output above, 2 comparisons are made to search for the target element, for both array and linked list.

Problem 2: Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

```
50  template<typename T>
51  void linearSearchArr2(int size, T dataFind, T data[]){
52      int total = 0;
53      for (int i=0; i<size; i++) {
54          if(data[i] == dataFind){
55              std::cout << "Element Found at Index: " << i << "\n";
56              std::cout << "Same Element Found: " << ++total << "\n";
57          }
58      }
59      if (total > 0){
60          return;
61      }
62      std::cout << "Element not Found.\n";
63  }
```

```cpp
template <typename T>
void linearSearchLL2(Node<T> *node, T dataFinding) {
    int total = 0;
    while(node != nullptr){
        if(node->data == dataFinding){
            std::cout << "Search was successful.\n";
            std::cout << "Same Element Found: " << ++total << "\n";
        }
        node = node->next;
    }
    if (total > 0){
        return;
    }
    std::cout << "Search was unsuccessful\n";
}
```

Output

```
ARRAY AND LINKED LIST LINEAR SEARCH
Element Found at Index: 1
Same Element Found: 1
Element Found at Index: 4
Same Element Found: 2
Search was successful.
Same Element Found: 1
Search was successful.
Same Element Found: 2
PS D:\Downloads (HDD)\CPE010\HOA6>
```

Problem 3: Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code

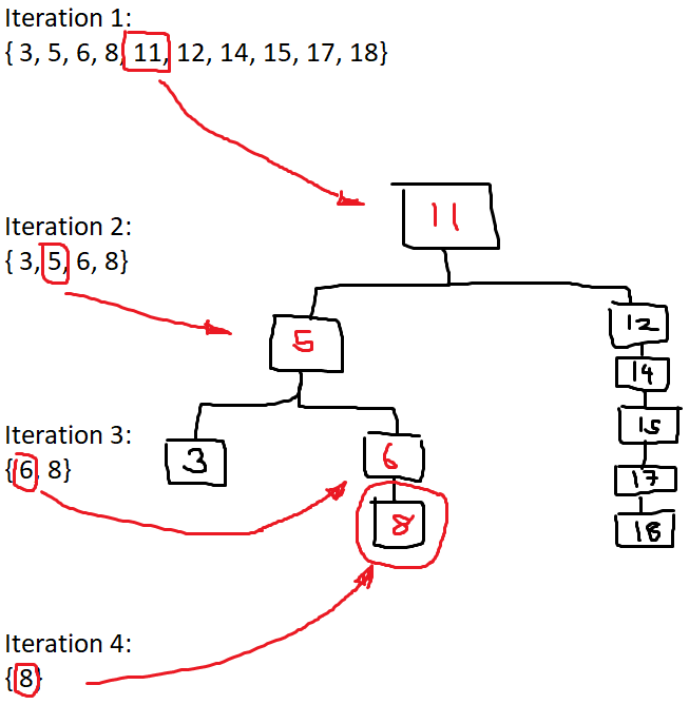
```
25      int dataset[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
26
27      binarySearchArr(8, dataset, max_size);
28      return 0;
```

```
81    template <typename T>
82    void binarySearchArr(T dataFind, T data[], int maxSize){
83
84        int low = 0, up = maxSize-1, middle;
85        while(low <= up){
86            middle = (int)((low+up)/2);
87            if(dataFind == data[middle]){
88                std::cout << "\nSearching is successful\n";
89                return;
90            }
91            else if(dataFind > data[middle]){
92                low = middle + 1;
93            } else {
94                up = middle - 1;
95            }
96        }
97        std::cout << "Search element was not found.\n";
98    }
99
100   template <typename T>
101   Node<T>* getMiddle(Node<T>* start, Node<T>* last){
102
103       Node<T>* slow = start;
104       Node<T>* fast = start;
105
106       while(fast != last && fast->next != last){
107           slow = slow->next;
108           fast = fast->next->next;
109       }
110       return slow;
111   }
```

Output

```
Searching is successful
PS D:\Downloads (HDD)\CPE010\HOA6\output>
```

"Draw a diagram that shows how the searching works per iteration of the algorithm."



Problem 4: Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code

```
25      int dataset[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
26
27      //binarySearchArr(8, dataset, max_size);
28      //return 0;
29      binarySearchArr2(dataset, 3, 18, 8);
30      return 0;
```

```cpp
template <typename T>
void binarySearchArr2(T data[], int low, int high, T dataFindings){
    if (low > high){
        std::cout << "No Data Found. " << std::endl;
        return;
    }

    int middle = (int)((low+high)/2);
    if (dataFindings == data[middle]){
        std::cout << "Element's Index Found at: " << middle << std::endl;
        return;
    }
    if (dataFindings < data[middle]){
        binarySearchArr2(data, low, middle - 1, dataFindings);
    } else {
        binarySearchArr2(data, middle + 1, high, dataFindings);
    }
    std::cout << "Element is not Found." << std::endl;
}
```

Output

```
Element's Index Found at: 3
```

Conclusion

For my conclusion for this activity, I've learned about 2 of the common searching algorithms from this lesson, which are Linear and Binary search. Linear searching, from the word linear, needs to compare each element one-by-one until you find your target element. Binary searching, starts from the middle to indicate where you should start searching from, is the target element greater or lesser than the middle index array size; this will continue to loop every iteration until the target element is found. The supplementary is complicated to most eyes, but with help from different blockmates, I was able to code the necessary task given and will be using this newly found knowledge in my future activities.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.